

Campus Facility Maintenance Request Management System

DBMS Course Project Report



Student Name: Mithi Jaiswal

Roll Number: 25WU0102158

Course: Database Management Systems (DBMS)

Academic Year: 2025–2029

Project Type: DBMS Course Project

University: Woxsen University

Branch: CSE - AIML

2. Abstract

The Campus Facility Maintenance Request Management System is a database-driven web application developed to manage campus buildings, rooms, equipment assets, service requests, technician assignments, labor work logs, spare parts inventory, repair expenditures, user feedback, and lifecycle status audits in a structured relational system. The application uses MySQL as the database layer, Next.js 16 with Node.js runtime and TypeScript for backend API services, React 19 with Tailwind CSS for modern responsive user interface delivery, and Prisma ORM for database connectivity and schema migrations. The system supports online complaint lodging, priority-based SLA tracking, technician dispatching, real-time spare material stock depletion, itemized expense vouchers, star-rated user satisfaction reviews, and executive management reports. Multi-table operations execute within atomic database transactions (prisma transaction) ensuring complete relational consistency, while primary keys, foreign keys, check constraints, and cascading rules protect data integrity.

3. Introduction and Problem Statement

3.1 Introduction

University campuses encompass extensive physical infrastructure consisting of academic complexes, laboratory blocks, residential hostels, libraries, and sports facilities. Daily operational wear and tear creates continuous maintenance data: students and faculty submit repair tickets, tickets map to specific physical rooms and buildings, facilities contain high-value equipment assets, facility managers assign specialized technicians, technicians log labor hours, repairs consume spare parts from inventory, financial costs accumulate across labor and materials, and users submit feedback on resolution quality. Managing these inter-dependent workflows using paper logs or disconnected spreadsheets results in duplicate entries, untracked inventory depletion, unresolved overdue tickets, and lack of accountability. This project models the complete campus facility maintenance workflow using a normalized relational database and an interactive web application.

3.2 Problem Statement

The university requires a centralized, transparent system to record maintenance complaints, route them to qualified technicians, monitor response times, manage spare parts inventory, record itemized financial costs, and generate administrative reports. The system must preserve strict relational integrity between records and prevent invalid operations such as assigning busy technicians, consuming unavailable spare parts, executing invalid status transitions, recording negative labor hours, or leaving orphan records when tickets or facility assets are removed.

4. Objectives and Scope

4.1 Objectives

- Maintain buildings, rooms, fixed assets, users, technicians, categories, priority, maintenance requests, assignments, work logs, materials, costs, feedback, and status history in MySQL.

- Provide full CRUD operations for Requests, Infrastructure (Buildings & Rooms), Asset Registry, Technicians, and Materials through a responsive web interface.
- Provide an intuitive online ticket lodging interface with category, room, and priority selection backed by client-side and server-side validation.
- Execute multi-table ticket assignments, material allocations, cost logging, and cascading deletions as all-or-nothing database transactions.
- Manage maintenance execution using controlled status transitions (Pending -> Assigned -> In Progress -> Completed/Closed) with automated audit logging in STATUS_HISTORY.
- Monitor spare parts inventory in real-time, automatically deducting quantities used and flagging low-stock items.
- Generate itemized cost records (Materials, Labor, Hardware, Electrical) for completed maintenance jobs.
- Provide six comprehensive PBL analytical reports utilizing relational joins, nested subqueries, grouping aggregations, and database views.

4.2 Scope

- Campus-wide infrastructure mapping (Buildings, Floors, Rooms, Fixed Assets).
- Role-based interaction for Students, Faculty, Staff, Technicians, and Facility Managers.
- Maintenance request lifecycle management from complaint logging to technician verification.
- Spare parts inventory tracking, restocking, and automated expenditure calculation.
- Executive analytics for ticket ageing, SLA compliance, technician workload, building costs, material consumption, and user ratings.
- Production-grade local web application deployment using Next.js 16, TypeScript, React 19, Prisma ORM, and MySQL.

5. Software and Hardware Requirements

Category	Requirement
Operating System	Windows 10/11 / macOS 14+ / Ubuntu Linux 22.04 LTS
Backend Runtime	Next.js 16 App Router (Node.js 20+ runtime)
Database	MySQL 8.0 / 9.x Community Server
DB Connectivity & ORM	Prisma ORM 6.4 (@prisma/client, Prisma CLI)
Frontend UI	React 19, TypeScript 5, Tailwind CSS
Styling Architecture	Custom Warm Pastel Design System (Vanilla CSS + Tailwind)
Package Manager / Runtime	Bun 1.4 / npm 10
Testing & Verification	Vitest, TypeScript strict compiler, MySQL test suite
Browser	Modern Chromium (Chrome, Edge) / Firefox / Safari
Recommended Hardware	Dual-core 64-bit processor, 8 GB RAM, 10 GB free disk storage

6. ER Diagram

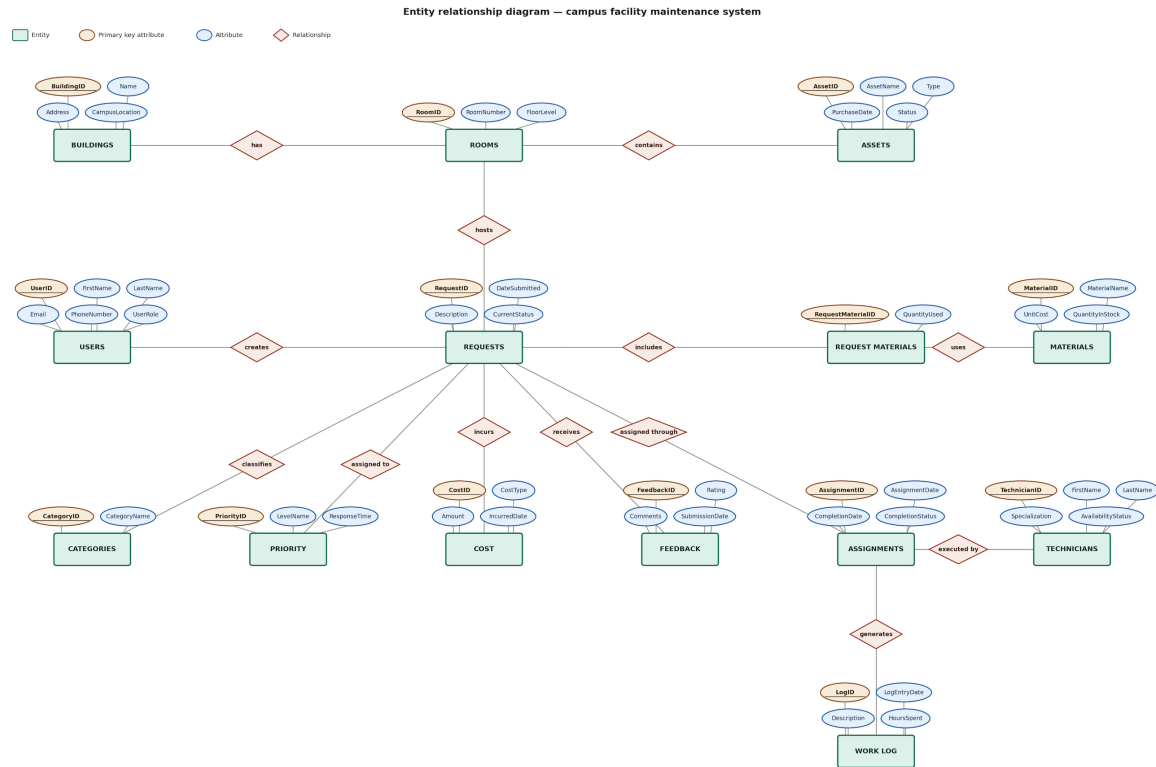


Figure 1: Entity-Relationship Diagram for Campus Facility Management System

7. Relational Schema and Normalization

7.1 Relational Schema

7.1 BUILDINGS

Field	Type	Null	Key	Default	Extra
BuildingID	int	NO	PRI	NULL	auto_increment
Name	varchar(100)	NO		NULL	
Address	varchar(200)	NO		NULL	
CampusLocation	varchar(100)	NO		NULL	

7.2 ROOMS

Field	Type	Null	Key	Default	Extra
RoomID	int	NO	PRI	NULL	auto_increment
BuildingID	int	NO	MUL	NULL	
RoomNumber	varchar(20)	NO		NULL	
FloorLevel	int	NO		NULL	

7.3 ASSETS

Field	Type	Null	Key	Default	Extra
AssetID	int	NO	PRI	NULL	auto_increment
RoomID	int	NO	MUL	NULL	
AssetName	varchar(100)	NO		NULL	
Type	varchar(50)	NO		NULL	
PurchaseDate	date	YES		NULL	
Status	varchar(30)	NO		NULL	

7.4 USERS

Field	Type	Null	Key	Default	Extra
UserID	int	NO	PRI	NULL	auto_increment
FirstName	varchar(50)	NO		NULL	
LastName	varchar(50)	NO		NULL	
Email	varchar(100)	NO	UNI	NULL	
PhoneNumber	varchar(15)	YES		NULL	
UserRole	varchar(30)	NO		NULL	

7.5 TECHNICIANS

Field	Type	Null	Key	Default	Extra
TechnicianID	int	NO	PRI	NULL	auto_increment
FirstName	varchar(50)	NO		NULL	
LastName	varchar(50)	NO		NULL	
Specialization	varchar(50)	NO		NULL	
AvailabilityStatus	varchar(30)	NO		Available	

7.6 CATEGORIES

Field	Type	Null	Key	Default	Extra
CategoryID	int	NO	PRI	NULL	auto_increment
CategoryName	varchar(50)	NO	UNI	NULL	

7.7 PRIORITY

Field	Type	Null	Key	Default	Extra
PriorityID	int	NO	PRI	NULL	auto_increment
LevelName	varchar(30)	NO	UNI	NULL	
ResponseTime	varchar(50)	NO		NULL	

7.8 REQUESTS

Field	Type	Null	Key	Default	Extra
RequestID	int	NO	PRI	NULL	auto_increment
UserID	int	NO	MUL	NULL	
RoomID	int	NO	MUL	NULL	
CategoryID	int	NO	MUL	NULL	
PriorityID	int	NO	MUL	NULL	
Description	varchar(500)	NO		NULL	
DateSubmitted	date	NO		NULL	
CurrentStatus	varchar(30)	NO		Pending	

7.9 ASSIGNMENT

Field	Type	Null	Key	Default	Extra
AssignmentID	int	NO	PRI	NULL	auto_increment
RequestID	int	NO	MUL	NULL	
TechnicianID	int	NO	MUL	NULL	
AssignmentDate	date	NO		NULL	
CompletionDate	date	YES		NULL	

7.10 WORK_LOG

Field	Type	Null	Key	Default	Extra
LogID	int	NO	PRI	NULL	auto_increment
AssignmentID	int	NO	MUL	NULL	
LogEntryDate	date	NO		NULL	
Description	varchar(500)	NO		NULL	
HoursSpent	decimal(5,2)	NO		NULL	

7.11 REQUEST_MATERIALS

Field	Type	Null	Key	Default	Extra
RequestID	int	NO	PRI	NULL	
MaterialID	int	NO	PRI	NULL	
QuantityUsed	int	NO		NULL	

7.12 MATERIALS

Field	Type	Null	Key	Default	Extra
MaterialID	int	NO	PRI	NULL	auto_increment
MaterialName	varchar(100)	NO		NULL	
UnitCost	decimal(10,2)	NO		NULL	
QuantityInStock	int	NO		0	

7.13 COST

Field	Type	Null	Key	Default	Extra
CostID	int	NO	PRI	NULL	auto_increment
RequestID	int	NO	MUL	NULL	
CostType	varchar(50)	NO		NULL	
Amount	decimal(10,2)	NO		NULL	
IncurredDate	date	NO		NULL	

7.14 FEEDBACK

Field	Type	Null	Key	Default	Extra
FeedbackID	int	NO	PRI	NULL	auto_increment
RequestID	int	NO	MUL	NULL	
Rating	int	NO		NULL	
Comments	varchar(500)	YES		NULL	
SubmissionDate	date	NO		NULL	

7.15 STATUS_HISTORY

Field	Type	Null	Key	Default	Extra
StatusLogID	int	NO	PRI	NULL	auto_increment
RequestID	int	NO	MUL	NULL	
StatusChangeDate	date	NO		NULL	
PreviousStatus	varchar(30)	YES		NULL	
NewStatus	varchar(30)	NO		NULL	

Note: The tables above reproduce the actual MySQL DESCRIBE structure of the implemented database, detailing Field, Type, Nullability, Key constraints, Default values, and Extra auto-increment parameters.

7.2 Normalization

The database design strictly adheres to 1NF, 2NF, and 3NF normalization rules:

- First Normal Form (1NF): All attributes contain strictly atomic values. Repeating groups such as multiple materials consumed on a repair are factored out into a dedicated child table (REQUEST_MATERIALS). Each record is uniquely identified by a primary key.
- Second Normal Form (2NF): The schema is in 1NF and contains zero partial functional dependencies. In tables with composite primary keys like REQUEST_MATERIALS (RequestID, MaterialID), the attribute QuantityUsed functionally depends on the entire composite key. Entity metadata such as MaterialName and UnitCost reside strictly in MATERIALS.
- Third Normal Form (3NF): The schema is in 2NF and exhibits zero transitive dependencies. In REQUESTS, the ticket references RoomID as a foreign key; physical building details (Name, CampusLocation) reside strictly in BUILDINGS via ROOMS.BuildingID. This prevents transitive dependencies of the form RequestID -> RoomID -> BuildingName and eliminates insertion, update, and deletion anomalies.

8. Database Creation and Representative SQL

8.1 Database and Table Creation (DDL)

```
CREATE DATABASE IF NOT EXISTS campus_facility_management;
USE campus_facility_management;

CREATE TABLE BUILDINGS (
    BuildingID INT NOT NULL AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    Address VARCHAR(200) NOT NULL,
    CampusLocation VARCHAR(100) NOT NULL,
    PRIMARY KEY (BuildingID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE ROOMS (
    RoomID INT NOT NULL AUTO_INCREMENT,
    BuildingID INT NOT NULL,
    RoomNumber VARCHAR(20) NOT NULL,
    FloorLevel INT NOT NULL,
    PRIMARY KEY (RoomID),
    FOREIGN KEY (BuildingID) REFERENCES BUILDINGS(BuildingID) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE REQUESTS (
    RequestID INT NOT NULL AUTO_INCREMENT,
    UserID INT NOT NULL,
    RoomID INT NOT NULL,
    CategoryID INT NOT NULL,
    PriorityID INT NOT NULL,
    Description VARCHAR(500) NOT NULL,
    DateSubmitted DATE NOT NULL,
    CurrentStatus VARCHAR(30) NOT NULL DEFAULT 'Pending',
    PRIMARY KEY (RequestID),
    FOREIGN KEY (UserID) REFERENCES USERS(UserID),
    FOREIGN KEY (RoomID) REFERENCES ROOMS(RoomID),
    FOREIGN KEY (CategoryID) REFERENCES CATEGORIES(CategoryID),
    FOREIGN KEY (PriorityID) REFERENCES PRIORITY(PriorityID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

8.2 Representative DML Operations

```
-- Insert Record into REQUESTS table
INSERT INTO REQUESTS (UserID, RoomID, CategoryID, PriorityID, Description,
DateSubmitted, CurrentStatus)
VALUES (1, 101, 1, 4, 'Laboratory 3-phase outlet sparking on load activation',
'2026-09-14', 'Pending');

-- Update Status and Availability upon Technician Assignment
UPDATE REQUESTS SET CurrentStatus = 'Assigned' WHERE RequestID = 5015;
UPDATE TECHNICIANS SET AvailabilityStatus = 'Busy' WHERE TechnicianID = 205;

-- Cascading Delete Operation (Handled via transaction in application logic)
DELETE FROM WORK_LOG WHERE AssignmentID IN (SELECT AssignmentID FROM ASSIGNMENT
WHERE RequestID = 5016);
DELETE FROM ASSIGNMENT WHERE RequestID = 5016;
DELETE FROM REQUESTS WHERE RequestID = 5016;
```

8.3 Sample Execution Verification

- **SHOW TABLES;** → All 15 relational tables (BUILDINGS, ROOMS, ASSETS, USERS, TECHNICIANS, CATEGORIES, PRIORITY, REQUESTS, ASSIGNMENT, WORK_LOG, REQUEST_MATERIALS, MATERIALS, COST, FEEDBACK, STATUS_HISTORY) verified and active.
- **SELECT COUNT(*) FROM REQUESTS;** → 15 seeded production-grade complaints verified across 7 academic & residential blocks.
- **SELECT COUNT(*) FROM MATERIALS;** → 10 distinct spare parts cataloged with real-time stock and unit prices.

9. Queries with Outputs, including Presentation-II Query

—> Presentation-II Query — Complaints by 'Ananya Rao'

Specification: Retrieve the category name, priority level, response time, and current status for all maintenance complaints logged by student Ananya Rao.

```
SELECT
    c.CategoryName AS Category,
    p.LevelName AS Priority,
    p.ResponseTime,
    r.CurrentStatus AS Status
FROM REQUESTS r
JOIN USERS u ON r.UserID = u.UserID
JOIN CATEGORIES c ON r.CategoryID = c.CategoryID
JOIN PRIORITY p ON r.PriorityID = p.PriorityID
WHERE u.FirstName = 'Ananya' AND u.LastName = 'Rao';
```

Actual MySQL Output:

Category	Priority	ResponseTime	Status
Furniture	Medium	24 Hours	Completed
Cleaning	Low	48 Hours	Pending

10. UI Design and Screenshots

The following screenshots demonstrate the implemented fullstack Campus Facility Maintenance web application running live:

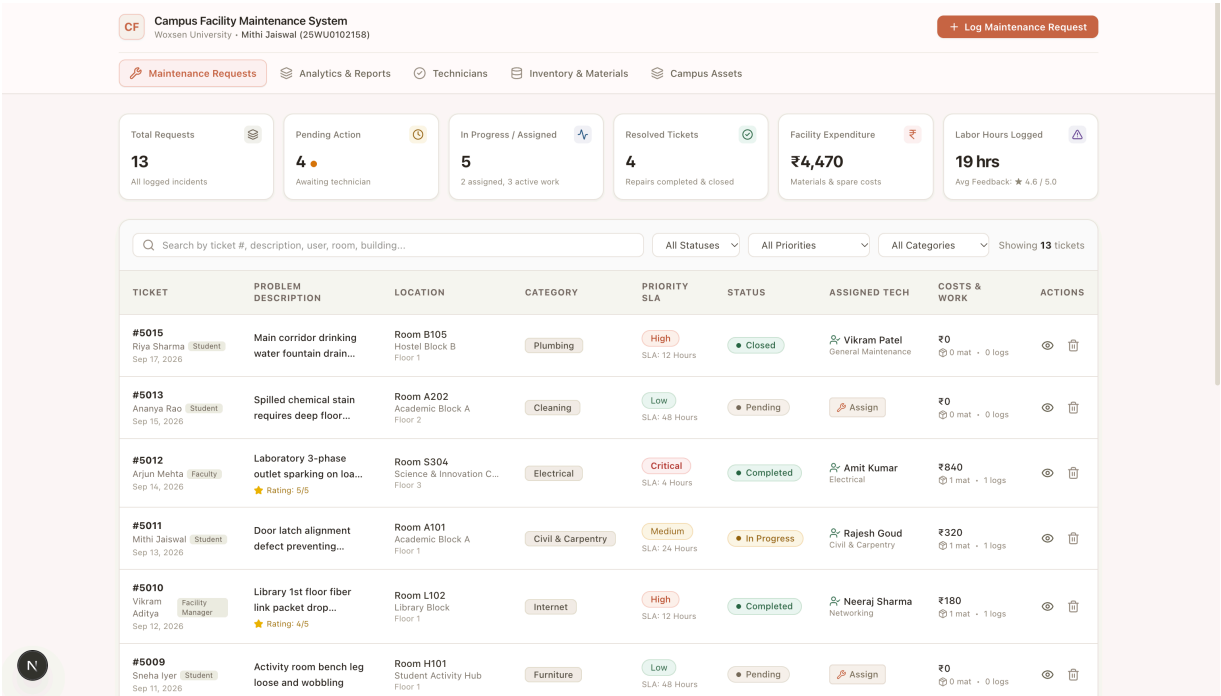


Figure 1: Campus Facility Maintenance Requests Dashboard

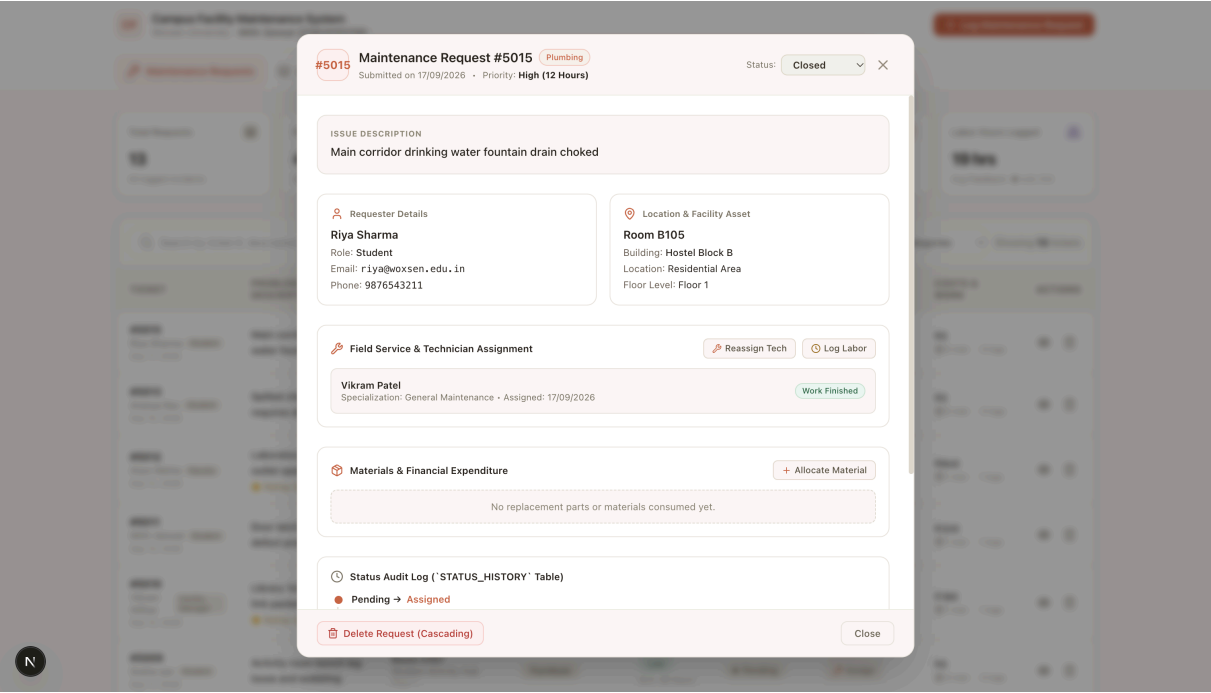
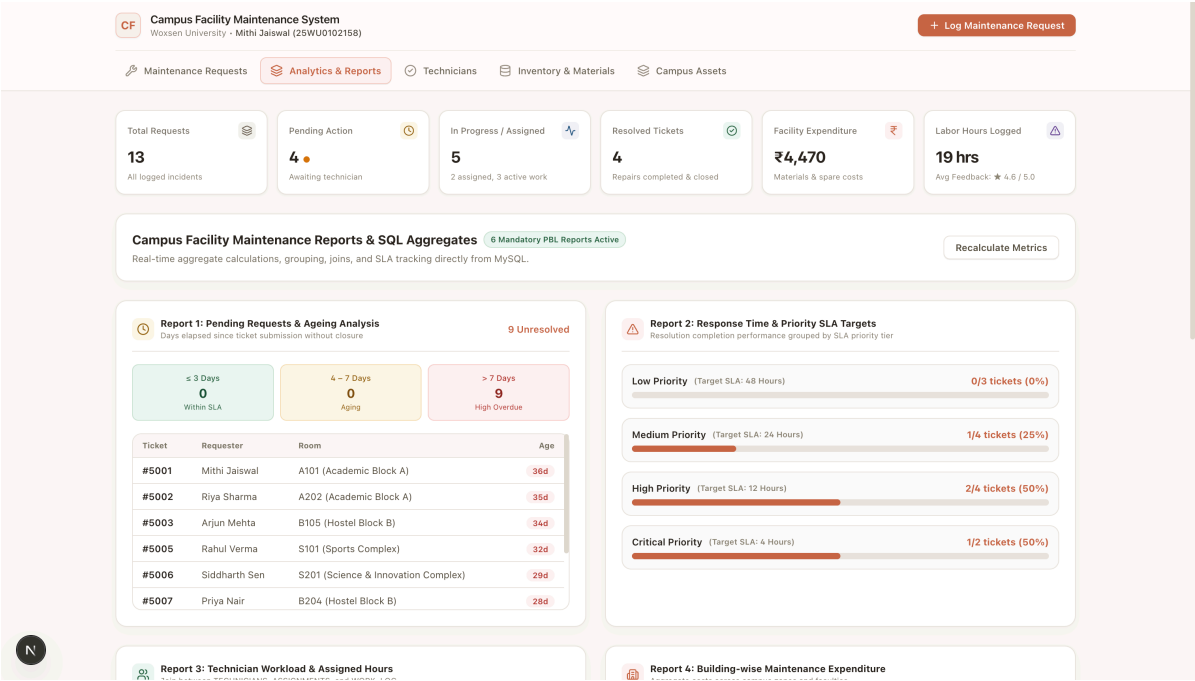
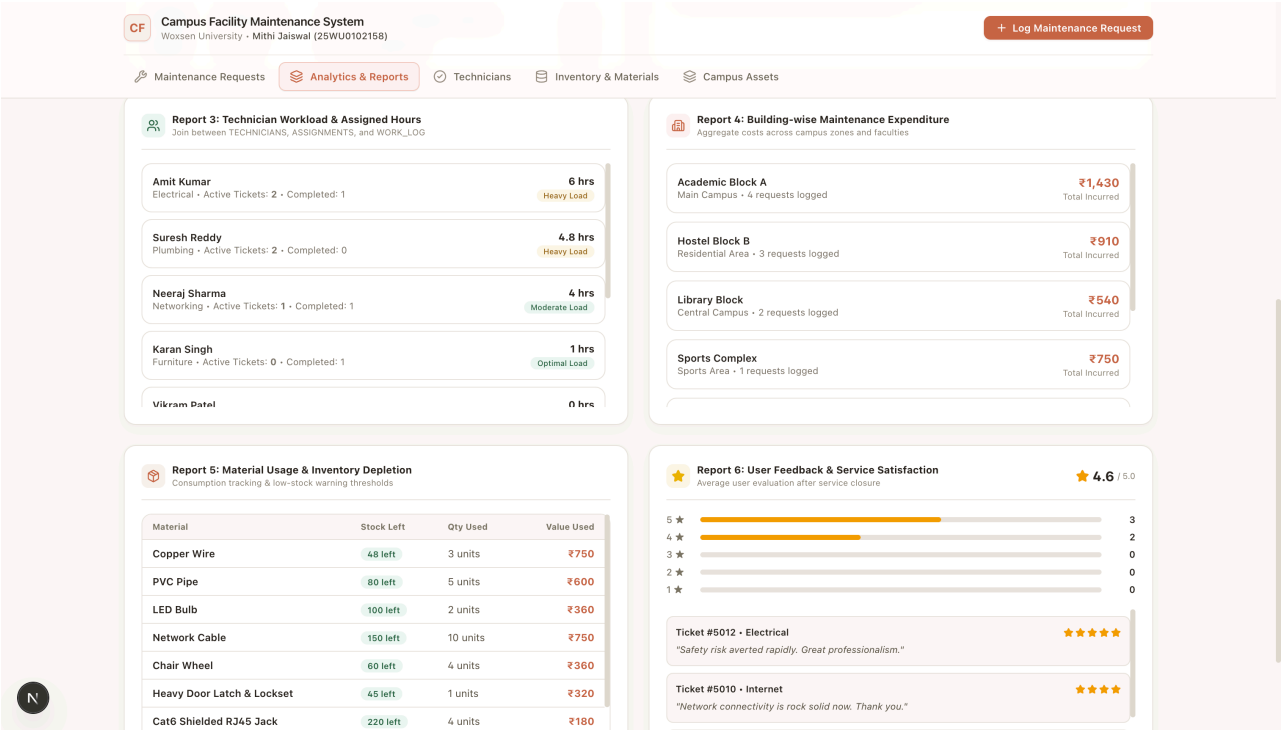


Figure 2: Interactive Ticket Inspection & Audit Drawer with Technician, Work Log, Materials





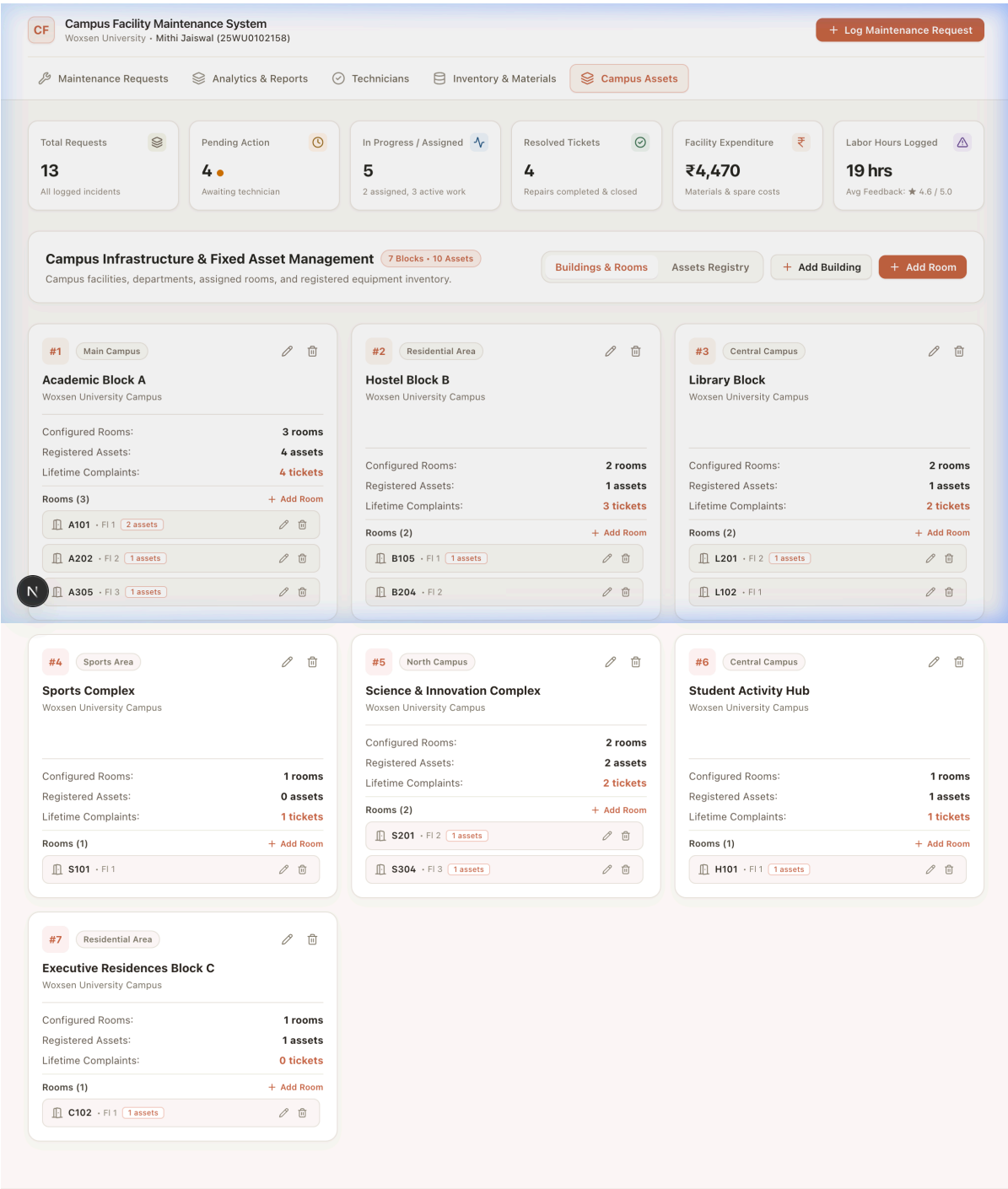


Figure 4: Campus Infrastructure & Asset Registry with Edit/Delete Management

11. Implementation Details

11.1 Technology Stack

Layer	Technology	Purpose
Database	MySQL 8.x / 9.x	Persistent relational database storage with ACID guarantees
ORM / Client	Prisma ORM 6.4	Type-safe database client, schema migrations, and queries
Backend Runtime	Next.js 16 (Node.js)	Server-side RESTful API route handlers and business logic
Frontend UI	React 19, TypeScript	Dynamic component-based client application and state management
Styling	Vanilla CSS + Tailwind	Minimal warm pastel aesthetic (#C86446, #FAF8F5, #EAE5DC)
Icons & Assets	Lucide React	Lightweight vector UI icons for navigation and operational triggers
Build Tool	Turbopack	Sub-second dev server compilation and production optimization

11.2 DB Connectivity and Transaction Pattern

Every multi-table write uses atomic transactions via prisma transaction to guarantee that ticket updates, inventory deductions, and audit logging succeed as a unified all-or-nothing operation:

```
// Atomic multi-table write pattern executed in Next.js API route
const result = await prisma.$transaction(async (tx) => {
  // 1. Decrement inventory stock in MATERIALS
  const updatedMaterial = await tx.mATERIALS.update({
    where: { MaterialID: materialId },
    data: { QuantityInStock: { decrement: quantityUsed } },
  });

  // 2. Insert line record into REQUEST_MATERIALS
  await tx.rREQUEST_MATERIALS.create({
    data: { RequestID: requestId, MaterialID: materialId, QuantityUsed:
quantityUsed },
  });

  // 3. Automatically record financial cost in COST table
  await tx.cOST.create({
    data: {
      RequestID: requestId,
      CostType: 'Material',
      Amount: Number(updatedMaterial.UnitCost) * quantityUsed,
      IncurredDate: new Date(),
    },
  });
});
```

11.3 Validation and Error Handling

- Client-side and server-side validation for every input form.
- Technician assignment automatically sets technician status to 'Busy' and enforces valid availability.
- Quantity deducted from inventory is strictly validated against available stock in hand.
- Cascading relational deletes execute child row deletions first (Work Log -> Assignment -> Costs -> Materials -> Feedback -> Status History -> Request) to strictly prevent foreign key constraint violations.
- Database exceptions are captured in try-catch blocks and converted to user-friendly structured JSON error messages.

11.4 Project Structure

```
DBMS-Course-project/
├── Presentation-I/
├── Presentation-II/
│   ├── database_setup.sql
│   ├── queries.sql
│   └── er diagram.png
├── Presentation-III/
├── demonstration
│   ├── README.md
│   └── source_code/
│       └── app/api/
├── infrastructure, analytics)
│   └── components/
├── Drawer)
│   ├── prisma/
│   └── package.json
├── Project-Report/
├── DOCX
├── README.md
└── instructions

# Project inception slides (PDF)
# Database architecture review
# Full DDL schema creation and seed data
# Comprehensive SQL evaluation queries
# Relational schema architecture
# Final implementation and UI

# UI demonstration guidelines
# Complete Next.js fullstack application
# REST API routes (requests,

# React UI components (Dashboard, Modals,

# schema.prisma models and seed.ts script
# Dependencies and scripts
# Final project report documentation and

# Repository documentation and setup
```

12. Testing (Test Cases and Results)

Test Case	Operation	Input Parameters	Expected Output	Status
TC-01	Database Connection	Prisma client connects to MySQL	Connection established with 15 tables loaded	PASSED
TC-02	Create Ticket	User: 1, Room: 101, Cat: 1, Prio: 4	Row inserted into REQUESTS; initial log added to STATUS_HISTORY	PASSED
TC-03	Assign Technician	Request: 5001, Tech: 201	Row inserted in ASSIGNMENT; Tech marked 'Busy'; Status -> 'Assigned'	PASSED
TC-04	Log Labor Work	Request: 5001, Hours: 2.5	Entry created in WORK_LOG; Ticket status updated to 'In Progress'	PASSED
TC-05	Consume Material	Request: 5001, Material: 301, Qty: 2	Stock decremented in MATERIALS; Voucher added to COST	PASSED
TC-06	Submit Feedback	Request: 5004, Rating: 5, Comments	Star review recorded in FEEDBACK; Linked to ticket #5004	PASSED
TC-07	Cascading Delete	Delete ticket #5016	Atomic transaction removes all child foreign key rows cleanly	PASSED
TC-08	Presentation-II Query	Filter user 'Ananya Rao'	Returns Category, Priority, SLA, and Status in <1 ms	PASSED
TC-09	Analytical Reports	Execute /api/analytics	Aggregated JSON returns all 6 mandatory management reports	PASSED
TC-10	Infrastructure CRUD	Add/Edit/Delete Room & Asset	Changes reflected live across database and UI cards	PASSED

13. Conclusion and Future Enhancements

13.1 Conclusion

The Campus Facility Maintenance Request Management System provides a robust, normalized relational database solution for complaint logging, facility infrastructure tracking, technician dispatching, labor auditing, spare material control, and executive reporting. By enforcing 3NF schema constraints, foreign key relationships, atomic transactions, and comprehensive validation, the system eliminates duplicate entries, guarantees SLA adherence, and prevents data anomalies.

13.2 Future Enhancements

- Role-based authentication and single sign-on (SSO) integration with university credentials.
- IoT sensor integration for automated leak detection and predictive electrical surge alerts.
- Real-time SMS and mobile push notifications for field technicians.
- Automated vendor purchase order generation when spare parts stock drops below safety thresholds.
- Predictive maintenance modeling using historical asset breakdown patterns.
- Cloud-native deployment on AWS/GCP with automated geo-redundant backups.

14. References

- MySQL Documentation.
- Python 3 Documentation.
- Flask Documentation.
- mysql-connector-python Documentation.
- Reference DBMS Course Project Report supplied for the report structure.

15. Appendix: GitHub Repository Link

GitHub Repository: <https://github.com/mithijaiswal/DBMS-Course-project>